Masaryk University
Faculty of Informatics

Static analysis of Python code

Bachelor's Thesis

Jakub Kiska

Brno, Spring 2021

Masaryk University
Faculty of Informatics

Static analysis of Python code

Bachelor's Thesis

Jakub Kiska

Brno, Spring 2021

This is where a copy of the official signed thesis assignment and a copy of the Statement of an Author is located in the printed version of the document.

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Jakub Kiska

Advisor: RNDr. Lukáš Němec

Acknowledgements

I would like to acknowledge the continuous support, help and lead provided by my advisor RNDr. Lukáš Němec throughout the course of the last eight months.

Abstract

Static analysis is one of several methods used for finding bugs in code. Many tools used today can perform static analysis on code written in Python. The primary goal of this thesis is to evaluate and compare five different tools used for this purpose. Every tool analyzed several test cases with known bugs and the selected open-source project. Particular attention has been paid to the number of reported issues, their severity, false positives, and user experience. The acquired results serve as a base for a final recommendation. Tools differed significantly. Therefore, the recommendation is split into several parts based on the preferences of a user.

Keywords

DeepCode, DeepSource, Pyflakes, Python, Pylint, SonarQube, static analysis

Contents

Introduction **1**

1 Introduction of used tools **3**
1.1 Pylint 3
1.2 Pyflakes 4
1.3 DeepSource 4
1.4 DeepCode 4
1.5 SonarQube 5

2 Test cases **6**
2.1 Invalid number of arguments 6
2.2 Ineffective logic formulas 7
2.3 Array access 7
2.4 Equals sign misuse 8
2.5 File manipulation 8
2.6 Possibility of division by zero 9
2.7 Invalid dictionary keys 10
2.8 Membership support 10
2.9 Style issues 11
2.10 Variable overflow 12
2.11 Recursion 12
2.12 Undefined variables 13
2.13 Undefined functions 14
2.14 Unreachable code 14
2.15 Function argument type 15
2.16 Changed type of variable 16
2.17 Possibly undefined variable 16
2.18 Unused or missing import 17
2.19 Other found issues 17
2.20 Execution times 17
2.21 Summary 19

3 Project testing **21**
3.1 Errors 21
3.2 Warnings and code smells 22
3.2.1 Pylint 23

3.2.2 DeepSource 23
3.2.3 SonarQube 24
3.2.4 DeepCode 25
3.2.5 Pyflakes 25
3.3 Style 25
3.3.1 Pylint 25
3.3.2 DeepSource 26
3.3.3 SonarQube 26
3.4 Execution times 27
3.5 Summary 27

4 General qualities **31**
4.1 Interface 31
4.2 Issue documentation 32
4.3 Community 32
4.4 Price 32

5 Recommendation **34**

Conclusion **35**

Bibliography **36**

A File to section mapping **39**

Introduction

Bugs in code pose a significant threat to the quality and reliability of the final product. There are many ways to improve quality. One such method is to perform a static analysis. The key feature of static analysis is the ability to analyze a given code with no input data and without executing code itself [1]. Many tools that perform static analysis are available for Python. These tools are subject to constant development because of the new versions of Python itself.

Because of the wide variety, it is not feasible to utilize every existing tool. Therefore, it is required to determine how similar or different their performances are and provide a recommendation on which tools should be used. This thesis compares well-established tools, including Pylint, Pyflakes and SonarQube, with more recent solutions, including DeepCode and DeepSource. DeepSource has been used in cooperation with NASA [2], and DeepCode implements a unique approach, including AI [3].

The first chapter introduces each tool and clarifies the process of their selection. The introduction of every tool incorporates statements from the respective documentation or official website. This indicates the focus of every tool.

The testing of each tool starts in the second chapter. The main goal of this chapter is to evaluate analyses of short pieces of code with known bugs. Every section focuses on a specific type of bug and how each tool reacted to it. All results are then evaluated and each tool ranked based on the number of fully correct, partially correct and false positive cases.

The testing continues in the third chapter. The chapter describes how an open-source project is selected and subsequently tested. Reported issues were numerable. Therefore, they were categorized into three sections. The first section focuses on errors, the second on warnings and code smells, and the last on style issues. Tools are compared using the number, type and severity of issues.

The fourth chapter compares the general qualities of the tools. These qualities mainly impact the overall user experience. Topics like interface, issue documentation, community threads and price are discussed.

The fifth chapter concludes the thesis with the final recommendation determined by the results from the previous chapters. Different solutions are provided based on team size and version control software usage.

1 Introduction of used tools

Let us begin with a brief introduction of the tools used for static analysis. The scope of various tools for Python may differ significantly. Some specialize in a specific problem [4], while others are designed to find multiple types of issues. The sheer variety of available candidates that can perform the static analysis resulted in a need for selection.

The first criterion for every tool was how universal the specific tool is. Choosing tools with a universal approach ensured two essential things. The tests were relevant for all tools, and the results of these tests were directly comparable. The selection of the first three tools was based on various articles about static analysis in Python [5, 6, 7]. Pylint, Pyflakes and SonarQube were selected as the most suitable candidates out of all tools mentioned in the articles. DeepCode was found thanks to a package available to Atom editor [8]. The innovative approach to static analysis with an AI made it a suitable tool for this thesis. The last tool is called DeepSource. The official site of this tool was found as a result of a Google search. This site claimed that DeepSource is used by NASA, Uber and Data Version Control [2]. Alleged cooperation with NASA sets a high standard for this tool and makes DeepSource a decent candidate for testing.

1.1 Pylint

Pylint was initially released in 2003 [9] and is under constant development. Based on the documentation, the tool can check for errors, seek code smells[1], enforce coding standards close to PEP8, scan for a certain type of errors, and offer suggestions related to refactoring and counts code's complexity. Pylint is run on a user's computer, and all files necessary are free to download through PIP. Version 2.6.0 was used.

1. A characteristic that possibly indicates a problem in code.

1.2 Pyflakes

The first version of Pyflakes was released in 2005 [10]. New versions are still being released to this day. It is designed to find various errors, work faster than Pylint or Pychecker[2], and avoid false positives. Nevertheless, the style of code is not checked by Pyflakes. It is run on a user's computer, and all files necessary are free to download through PIP. Version 2.2.0 was used.

1.3 DeepSource

DeepSource was founded in 2018 [2]. The main focus of this tool is to automate all objective parts of the code review, analyze code with the rate of false positives less than 5%, automatically fix the code with integrated Autofix[3], and transform the style with various transformers like Autopep8 and Black. DeepSource is compatible with Bitbucket, GitHub, GitLab, Travis CI and CircleCI. In addition to Python, this tool supports programming languages like Go, Ruby, JavaScript, and more. The results of this tool listed in this thesis are from March to May of 2021[4].

1.4 DeepCode

DeepCode utilizes AI that learns from open-source repositories [11]. The two main features of this tool are near to real-time feedback and machine learning approach. The developers claim that DeepCode is up to 54 times faster than comparable tools. Supported languages are Javascript, Java, Typescript, Python, C++ and C. DeepCode can be integrated into IDE, GitHub, Bitbucket, GitLab and various custom extensions and integrations. A user can also request a free trial for a fully self-hosted Docker container. The results of this tool listed in this thesis are from March to May of 2021[5].

2. A static analysis tool for Python 2.0 through 2.7.
3. A tool used for the automatic fixing of primitive bugs.
4. The time frame is used in this case because the website did not provide the specific version.
5. See footnote 4.

1.5 SonarQube

The development of the Sonar platform started in 2007 [12]. The community edition used for this thesis provides static code analysis for 15 languages, detects bugs and vulnerabilities, reviews security hotspots, tracks code smells, fixes user's technical debt, and provides code quality metrics and history [13]. Commercial editions can also be fully integrated into GitHub, GitLab, Bitbucket and more. The community edition is limited to importing repositories and authentication [14]. In addition, the community edition requires local hosting of a SonarQube server. Version 8.7 was used.

2 Test cases

The following sections examine the results gathered from static analyses of the prepared test cases. They focus on whether a tool discovered a specified error or resulted in a false positive or a false negative. Furthermore, the speed of execution, how well it followed standards and comparison with other tools are included. All results are summarized in Table 2.1. Used test files are attached to the thesis archive and the mapping of files to the sections can be found in Appendix A.

2.1 Invalid number of arguments

The first set of tests verified if selected tools are able to recognize an incorrect number of arguments provided to a function. This test covers user-defined function called with three arguments instead of two and built-in functions including *abs* called with two arguments, *any* called with one or zero arguments, *bin* called with two arguments, and *range* with no arguments.

Pylint managed to recognize the problem in user-defined function and reported *E1121: Too many positional arguments for function call* with the corresponding line from the source file. On the other hand, it ended with a false negative in the case of built-in functions. Furthermore, running static analysis on *range* with an invalid number of arguments results in Pylint's crash.

Pyflakes reported no problems in user-defined and built-in functions. This made both results false negatives.

The following tools, DeepSource and DeepCode, detected a higher number of arguments in the mentioned user-defined function. Like the previous two tools, DeepSource and DeepCode could not find a single problem related to built-in functions with incorrect arguments.

The last tested tool, SonarQube, achieved the best result even with the default set of rules[1]. This tool was able to find the wrong amount of arguments in user-defined function as well as in built-in functions.

1. A rule defines a specific type of problem and can be turned on and off in the settings.

All problems with functions *abs*, *any*, *bin* were found by SonarQube. The only false negative occurred on function *range* with no arguments.

To summarize this set of tests, Pyflakes was unable to detect an invalid number of arguments. Pylint, DeepSource and DeepCode demonstrated an ability to detect an invalid number of arguments in user-defined function but not in built-in functions. The most successful tool was SonarQube. It found almost every occurrence of an invalid number of arguments. The only exception being the function *range* without arguments, which was left unnoticed.

2.2 Ineffective logic formulas

The following part is focused on discovering ineffective logic formulas in functions. The mentioned test case consists of a function:

```
    sub1 = not arg1
    sub2 = not sub1 or arg2
    sub3 = not sub2 and arg2
    sub4 = not arg1 and sub3
    return sub4
```

Listing 2.1: Ineffective logic formula

The formula represented by Listing 2.1 always returns *False* for every possible combination of arguments.

All five tools overlooked this inefficiency and suggested nothing in regard to simplifying the function.

2.3 Array access

The next part of the testing was constructed to test whether a tool does check array boundaries or not. The test itself consisted of a function that asked a user for an undefined number of variables to store in an array. Afterwards, the array was handed to a loop that was used to print all items from that array and one behind the last one. This is a frequent problem called off-by-one error, and it causes *IndexError: list index out of range* whenever the piece of code with this problem is run.

All five tools ware not able to recognize this problem in the tested piece of code.

2.4 Equals sign misuse

Misusing equals sign and double equals sign may be a common problem for an inexperienced programmer. This test was suited to find out if the tested tool can find this issue. The tested piece of code included the following line:

```
11     if array = []:
```

Listing 2.2: Equals sign misuse

Pylint, Pyflakes and DeepSource found and correctly reported the issue. SonarQube overlooked this problem with the default set of rules. When the analysis was redone with all rules enabled, SonarQube registered and reported the mentioned problem. The only unsuccessful tool was the DeepCode because the analysis ended with a false negative result.

2.5 File manipulation

The next test focused on another common problem related to reading a file after it was already closed. A function opened a file, and then the file was passed as an argument to another function that closed it if the file contained the string "skip". After that, the first function tried to read the content of a file. Therefore this creates the possibility of reading a closed file. For testing purposes the opened file contained string "skip" and running this piece of code resulted in: *ValueError: I/O operation on closed file.*

Pylint, Pyflakes, DeepCode and SonarQube overlooked this issue and reported no problems with the tested piece of code.

Even though DeepSource failed to report the problem specifically, it offered a solution to prevent reading a file after it was closed. The suggested solution was to use *with* statement instead of function *open*. This implementation would prevent the flaw mentioned earlier because the file is automatically closed at the end of a block and does not need to be closed explicitly.

2.6 Possibility of division by zero

This chapter's topic is how well a tool can predict that a program can come to a point where it needs to divide by zero. Testing consisted of three different code fragments.

```
def division_by_zero_error(parts, radius):
    ...
    return math.pi * (radius ** 2) / parts

...
division_by_zero_error(0, 10)
```

Listing 2.3: Certain division by zero

The first code fragment included a simple formula for counting an area of a circle and then dividing it into pieces. An argument determined the number of pieces. Limit values for argument were not defined. Therefore the usage of zero was not restricted. This function is called with zero as an argument. Thus division by zero is inevitable for this specific test. On the other hand, if the function is called with a different number, it will function properly.

```
return math.pi * (radius ** 2) / 0
```

Listing 2.4: Certain division by zero

The second code fragment is a slightly modified version of the first one. Instead of using an argument as a variable for dividing the counted area, it was replaced by zero. Thus this line will never execute without an error.

```
    num = input_control()
    return math.pi * (radius ** 2) / num
```

Listing 2.5: Possible division by zero

The last code fragment replaced the zero from Listing 2.4 by an input function. This means that a division by zero may happen if a user decides to input zero, but it is unknown until the program approaches that line.

The results of all tested tools were insufficient. They provided no information about a possible division by zero when tested on the mentioned three code fragments. Surprisingly the code fragments in Listing 2.4 with a clear case of division by zero went unnoticed.

2.7 Invalid dictionary keys

This part tested how well a static analysis tool can cope with finding a call with an invalid dictionary key.

```
def key_error(code):
    ...
    subjects = {
        "ib107":"Computability and complexity",
        "pb007":"Software engineering",
        "pb154":"Dabatase systems basics",
        "pv028":"Applied Information Systems",
        "pv169":"Data transfer basics"}
    return subjects[code]

...
key_error("pv028")
key_error("pv027")
```

Listing 2.6: "pv027" is not in *subjects*

The last line in Listing 2.6 includes a key not listed in the dictionary above.

All the tools that have undergone testing overlooked the invalid key since they reported no problems.

2.8 Membership support

In this section, the ability to decide if a variable supports a membership test will be discussed. There were two code fragments tested for this purpose.

```
    zero_array = [0]
    return '0' in zero_array[0]
```

Listing 2.7: Invalid code

Listing 2.7 includes a line which tries to access an integer as an array.

```
    zero_array = [0]
    zero_array[0] = "00"
    return '0' in zero_array[0]
```

Listing 2.8: Valid code

Listing 2.8 is a variation of Listing 2.7. It fixes the problem of accessing zero as an array and replaces zero with a string containing two zeroes. Therefore the last line always returns *true*. This was done to create potential false positives.

The results varied in this section. Pylint managed to identify the problem in Listing 2.7. On the other hand, it reported the same problem with Listing 2.8. This indicates that while it can distinguish a problem of this kind, it may result in a false positive.

Pyflakes, DeepSource, DeepCode and SonarQube reported no problems with both listings, meaning that results of testing the code fragment in Listing 2.7 were false negatives.

2.9 Style issues

The next test evaluated the control of elementary PEP 8 guidelines. The first one being the usage of docstring and the second was trailing whitespaces.

In the case of Pylint, the result was mixed since it found the missing docstrings but just two out of four cases of trailing whitespaces. In contrast, Pyflakes and DeepCode revealed no problems. The remaining two tools, DeepSource and SonarQube, detected no problems with their default settings. Nevertheless, both of these tools offer style checking in their settings. After enabling these setting, both of these tools found missing docstrings and all trailing whitespaces. In addition to these findings, both tools reported other style-related issues. SonarQube outlined the use of *pass* within loops, commented out code in the test cases, and one case where comments were not dense enough[2]. DeepSource also reported the use of *pass* within loops. Moreover, DeepSource proved to be the most strict and required two blank lines after the end of function or class, two blank lines between functions and classes, whitespace after ',', ';', or ':', block comments to start with one space before the pound sign '#' and the comment itself, one blank line between the methods of a class, and a continuation line to be indented.

2. SonarQube has the default minimal value set to 25% comment density.

2.10 Variable overflow

Even in Python, floats have their limits. This section focuses on how well a tool can detect that the limit was exceeded.

```
17     result = sys.float_info.max ** 2
```

Listing 2.9: Float value overflow

Calling the line of code represented by Listing 2.9 results in *OverflowError*.

The results of selected tools were rather underwhelming because all of them failed to discover this problem in the tested piece of code.

2.11 Recursion

The main goal of this section is to focus on a recursion. It consists of two common bugs related to recursion and one case of recursion in a class definition.

```
6  def recursion_error3(var, base):
...     ...
10     if var == 0:
11         return base
12     base = base * 2
13     return recursion_error3(var, base)
```

Listing 2.10: Function does not decrease argument

The first type of issue is a function recursively called with the same argument. This causes Python to exceed maximum recursion depth.

```
15 def recursion_error2(var, base):
...     ...
19     base = base * 2
20     return recursion_error2(var - 1, base)
```

Listing 2.11: No terminating case

The second type is a recursive function with a decreasing argument, but without a terminating case that would break the recursion chain.

```
14     def __str__(self):
15         return str(self)
```

Listing 2.12: Class method recursion

The last type is recursion in a method of a class. The method has no terminating case and does not change the argument.

These test cases proved to be challenging for static analyses. Pylint, Pyflakes, DeepSource and DeepCode ended with false negatives in all three cases. SonarQube with default settings did better than most, but to a certain extent, as it was just partially successful. SonarQube reported the code fragment represented by Listing 2.11. The report suggested adding a way to break out of recursion, which was precisely the problem.

2.12 Undefined variables

Usage of an undefined variable was the main focus of the following test. Two undefined variables were used to make the test more extensive. The context differs for both variables.

```
12      while var > (und1 + var2):
13          if und1 < var2:
14              und1 += var2
...     ...
18      und1 = 0
```

Listing 2.13: Undefined variable *und1*

```
16      und2 += und1
```

Listing 2.14: Undefined variable *und2*

The first undefined variable called *und1* is used multiple times before its initiation. On the other hand, the second undefined variable called *und2* is only used as a variable that is being incremented.

After conducting the previous tests, most of the tools proved that they could recognize undefined variables. Among the successful tools were Pylint, Pyflakes and DeepSource. Mentioned static analysis tools reported both variables for being unused. On the contrary, static analyses with SonarQube and DeepCode ended with false negatives in both cases.

2.13 Undefined functions

In addition to undefined variables, an undefined function can also cause a significant problem. This section tested two forms of an undefined function.

```
undefined_error()
defined_later_error()

def defined_later_error():
```

Listing 2.15: Undefined and later defined functions

In the first case, the called function is not defined in the piece of code. On the contrary, the other case uses a function that is defined. Calling the latter function causes an error because the function is defined after the mentioned call.

Results differed substantially. As the most problematic issue to detect turned out to be the latter function. Function *defined_later_error* was found only by Pyflakes. Pyflakes, as the only tool, disclosed that both functions were problematic. Testing with Pylint, DeepSource and SonarQube revealed the first form of the undefined function. SonarQube accomplished this only after allowing all rules in the options. DeepCode overlooked both issues.

2.14 Unreachable code

Another topic related to undefined function is unreachable code. The tested piece of code included two different kinds of unreachable code.

```
    if length > 0 or length < 100:
        ...
        return None
    return unreachable_2(length)
```

Listing 2.16: Always true condition

The function called *unreachable_2* is referenced in the piece of code but the condition before it is always true. Therefore, the *unreachable_2* will never be called. Unlike *unreachable_2* the second unreachable function named *unreachable_3* is never called.

Testing the piece of code in Listing 2.16 with Pylint, Pyflakes, DeepSource, DeepCode and SonarQube resulted in false negative because

all tools could not identify *unreachable_2* or *unreachable_3* as an unreachable code.

2.15 Function argument type

This section tested how well a tool can keep track of types of arguments in a function. The testing sample consisted of two pieces of code. The main difference between these two pieces was how the variables are initiated. The function includes *len* function which returns *int* value. Then a function *remove* is called on this object with type *int*. Running this code fragment results in *AttributeError*.

```
5  def attribute_error(array, acc):
...     ...
9       arr_len = len(array)
10      arr_len.remove(acc)
11
...
13 attribute_error([0, 1, 2, 3], 2)
```

Listing 2.17: Variables passed as an argument

```
5  def attribute_error():
...     ...
9       array = [0, 1, 2, 3]
10      acc = 2
```

Listing 2.18: Variables initialised in the function

The presented piece of code in Listing 2.18 includes only the part where it differs from the piece of code in Listing 2.17. Lines 2 to 5 from Listing 2.17 are present in the file represented by Listing 2.18.

DeepCode outperformed the competitors in this regard. Both tested pieces of code were appropriately marked as problematic. While Pylint and DeepSource managed to find the problem in Listing 2.18, they failed to discover the identical problem in Listing 2.17. This implies that the different initialization of variables caused the difference. On the contrary, Pyflakes and SonarQube overlooked this problem in both pieces of code.

2.16 Changed type of variable

Following the previous section, this part continues to discover how well a tool can estimate the type of a variable. The static analysis tools were tested on a on a code fragment that included a list with items of type *int* and *str*.

```
    current = 1992
    for whats_next in ["1.1.1993", 1994]:
        if current == str(current):
            print(current[0:4])
        else:
            print(current)
        current = whats_next
```

Listing 2.19: Variable named current is changed with every iteration

As seen in Listing 2.19, the loop changes the value of variable *current* with every iteration.

As the testing finished, Pylint and DeepCode ended up being the least successful tools. These analyses ended with a false positive. Both reported that variable *current* is unsubscriptable. On the contrary, Pyflakes, DeepSource and SonarQube reported no problems with the tested code fragment.

2.17 Possibly undefined variable

This section focuses on discovering possibly undefined variables.

```
    for item in iterable:
        pass
    return item
```

Listing 2.20: Problematic cycle

The piece of code from Listing 2.20 may be problematic when the iterable object has no items. Then the *for* loop is skipped and variable *item* remains undefined.

Pylint and DeepSource succeeded in finding the problematic variable and reported it as a possibly undefined loop variable. Pyflakes, DeepCode and SonarQube were not as successful and reported no problems.

2.18 Unused or missing import

Looking for unused or missing imports may be quite useful when a programmer deals with a bigger project. The selected tools were tested on a piece of code that included one unused import (*gc*) and a method of a missing import (*math*).

Pylint and Pyflakes managed to find the unused import and *math* was marked as an undefined variable. DeepSource detected only *math* as an undefined name. DeepCode and SonarQube with default rules proved to be unsuccessful in this task. After activating more rules, SonarQube performed better and marked *math* as not defined.

2.19 Other found issues

In addition to the flaws mentioned earlier, SonarQube reported other problems too. The analysis with default set of rules reported that return values of functions *abs* and *bin* must be used. These two functions were discussed in Section 2.1.

A full set of rules discovered even more issues. The first and most numerable type of problem was that tests covered less than 65% of lines. A class not inheriting from *object* or some other new-style class was the second reported problem.

2.20 Execution times

Execution times can be split into two categories based on whether the tool is installed and run on the user's computer or not. If so, the duration can be measured with greater precision[3]. In addition to the mentioned precision, most of these tools create an opportunity to analyze all pieces of code separately as well as to test the whole directory.

```
$j=0; For ($i=0;  $i -le 100; $i++) {
    $j = $j + (Measure-Command {pylint example.py}).
   TotalMilliseconds
}; $j = $j / 100; echo $j
```

Listing 2.21: PowerShell code fragment used for time measuring

3. The time was not measured with exact precision but rather as an orientational element.

The piece of code in Listing 2.21 runs a specified static analysis on a Python file one hundred times and calculates an average length of a single test.

Let us start with the tools installed on the user's computer. When the files were tested separately the results for SonarQube with default rule set were in range from 12.1 seconds[4] to 12.5 seconds. Measured values for the full set of rules were similar and ranged from 12.0 seconds to 12.6 seconds. The only notable exception was a piece of code that included a syntax error. This file was analyzed in 8.9 seconds regardless of the rule set. Pylint performed a lot faster. The fastest time was 0.6 seconds and the slowest was 0.9 seconds. When compared to the previous two, Pyflakes had the fastest execution times. These times reached 0.2 seconds for all test cases. When combined, the full run-time was 4.3 seconds for Pyflakes, 17.5 seconds for Pylint, 291.8 seconds for SonarQube with default rule set and 291.4 seconds for SonarQube with all rules enabled.

Based on these tests testing the whole directory appeared to be more than ten times as time efficient as testing the each file separately. The measured times for the whole directory were 0.2 seconds for Pyflakes, 1.5 seconds for Pylint, 13.5 seconds for SonarQube with default rule set and 13.4 seconds for SonarQube with all rules enabled.

The remaing two tools, DeepCode and DeepSource, are available through the web-based interface. The excecution times are therefore based on the data provided in the report. DeepSource was the slower tool with the time reaching 11.0 seconds. In contrary, DeepCode finished its static analysis in 3.0 seconds. Despite the good time, DeepCode had a different time related problem. All projects are placed in a queue before being analyzed. The times in queue differed substantially. The fastest time was less then one second and the longest time was more than one hour. However, in most cases the queue time was under a minute.

In summary, static analyses are faster when run on a whole directory and not on every file separately. The fastest tool was Pyflakes, the second was Pylint, the third was DeepCode, the fourth was DeepSource, and the slowest was SonarQube.

4. All listed times are rounded to one decimal place.

2.21 Summary

All results can be found in Table 2.1. Altogether, the best result was achieved by DeepSource. This tool achieved five fully correct results, four partially correct results and zero false positives.

The second place belongs to Pylint and Pyflakes. Pylint detected at least one problem in more test cases than Pyflakes, but Pyflakes accomplished a higher number of fully correct results than Pylint. Pyflakes achieved five fully correct results and no false positives. As the only tested tool, it discovered all undefined functions in Section 2.13. In comparison, Pylint achieved four fully correct results, five partially correct results and two false positives. As the only tested tool, it discovered a membership support error in Section 2.8.

Subsequently, SonarQube found fewer problems than Pylint. The following results for SonarQube are based on the rule set with all rules enabled. This tool achieved three fully correct results, four partially correct results and no false positives. As the only tested tool, it discovered at least one problem with the recursion in Section 2.11.

Finally, the worst results were accomplished by DeepCode. This tool achieved one fully correct result, one partially correct result and one false positive. Nevertheless, as the only tested tool, it discovered all problematic argument types in Section 2.15.

Table 2.1: Test cases results

Chapter	Pylint	Pyflakes	DeepSource	DeepCode	SonarQube default rules	SonarQube all rules
2.1 Invalid number of arguments	PC (1/6)	FAIL	PC (1/6)	PC (1/6)	PC (5/6)	PC (5/6)
2.2 Ineffective logic formulas	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
2.3 Array access	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
2.4 Equals sign misuse	OK	OK	OK	FAIL	FAIL	OK
2.5 File manipulation	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
2.6 Possibility of division by zero	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
2.7 Invalid dictionary keys	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
2.8 Membership support	PCFP (1,1/2)	FAIL	FAIL	FAIL	FAIL	FAIL
2.9 Style issues	PC (4/6)	FAIL	OK	FAIL	FAIL	OK
2.10 Variable overflow	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
2.11 Recursion	FAIL	FAIL	FAIL	FAIL	PC (1/3)	PC (1/3)
2.12 Undefined variables	OK	OK	OK	FAIL	FAIL	FAIL
2.13 Undefined functions	PC (1/2)	OK	PC (1/2)	FAIL	FAIL	PC (1/2)
2.14 Unreachable code	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
2.15 Function argument type	PC (1/2)	FAIL	PC (1/2)	OK	FAIL	FAIL
2.16 Changed type of variable	FP	OK	OK	FP	OK	OK
2.17 Possibly undefined variable	OK	FAIL	OK	FAIL	FAIL	FAIL
2.18 Unused or missing import	OK	OK	PC (1/2)	FAIL	FAIL	PC (1/2)

OK = fully correct, PC (x/y) = partialy correct (x = correct subtests, y = all subtests), PCFP (x,y/z) = partially correct with some subtests being false positive (x = correct subtests, y = false positives, z = all subtests), FP = false positive, FAIL = fully incorrect.

3 Project testing

After the simple test cases, the focus of this thesis shifted to a larger open-source project. The requirements for the project were that it should be open-source, written entirely in Python and consist of approximately 20,000 lines of code. A candidate was then found on GitHub. This project focused on parsing, manipulating, and generating BridgePoint executable UML models, also called xtUML models. According to GitHub the project is entirely written in Python [15]. The number of lines exceeds 21,000 lines written in Python. The project works with Python 2.7 or 3.0 [16]. Version 2.1.0 of the project was tested [17]. The acquired results were split into three categories based on their severity. Warnings and code smells may be difficult to distinguish. Therefore, they were merged into one category.

3.1 Errors

Pylint and DeepSource reported the most major problems. These problems included import errors, instance having no member with the required name, no value for a parameter, and objects not being callable. The remaining three tools reported no problem that can be classified as an error.

Import errors were found only by Pylint. There were five import errors in total. Module *ply* was correctly recognized as problematic since it is a requirement for this project [16], and it was not installed before the analysis. This module was mentioned in four separate files. The last remaining error was related to current package import in *bridgepoint\gen_sql_schema.py*. The cause of this problem was not further specified.

An instance having no member with required name was found only by Pylint. There were four cases of this problem reported in *bridgepoint\prebuild.py*. Class *ActionPrebuilder* references member *act_act* in two methods. Although the class *ActionPrebuilder* has no such member, derived classes *BridgePrebuilder*, *FunctionPrebuilder*, *OperationPrebuilder*, *TransitionPrebuilder*, *DerivedAttributePrebuilder*, *RequiredOperationPrebuilder*, *RequiredSignalPrebuilder*, *ProvidedOperationPrebuilder* and *ProvidedSignalPrebuilder* have the mentioned member defined. Therefore,

the problem only occurs when the method is called from an instance of *ActionPrebuilder* class and not an instance of a derived class. Based on further use, this can be classified as a false positive.

Pylint and DeepSource found a problem specified as *no value for a parameter*. Both tools reported eight missing arguments in one call.

```
122 class CreateAssociationStmt(Stmt):
123     def __init__(self, rel_id, source_kind,
    source_cardinality, source_keys,
124                  source_phrase, target_kind,
    target_cardinality, target_keys,
125                  target_phrase):
...         ...
617         args = [p[4]]
618         args.extend(p[6])
619         args.extend(p[8])
620         p[0] = CreateAssociationStmt(*args)
```

Listing 3.1: Sequence in *xtuml\load.py* causing no value for parameter report

The piece of code in Listing 3.1 is valid if *p* is a list with a specific structure of variables. Without using that structure, the call will result in an error. The number of arguments provided by *args* is difficult to determine without deeper knowledge of the remaining parts of the project.

The last type of error is *calling a non-callable object*. DeepSource reported a class method *self.readfunc* in *xtuml\tools.py*. In addition to *self.readfunc*, Pylint also reported *self.next* in the same class. Although the class *IdGenerator* has an attribute with the specified name, derived classes *UUIDGenerator* and *IntegerGenerator* implement the called method. Therefore, *IdGenerator* might have been intended as an abstract class because the reported call is in the constructor itself. The same applies to the second reported method. If an iterator of a derived class is instanced and the method is called, it does not result in an error.

3.2 Warnings and code smells

Results differed significantly in this category. While DeepSource found the most issues, Pylint found the most types of different issues.

3.2.1 Pylint

The total number in this category reached 614 reported problems of 34 different types for Pylint.

The most numerable was a method that could be function, redefined argument with a local name, too few public methods, unused argument, *elif* condition after *return* statement, useless inheritance from *object* class, logging not being lazy, inconsistent return statements, and similar lines in different files.

Another problem was exceeding a user-definable maximum value for ancestors, public methods, local variables, statements, branches, return statements and instance attributes. Two deprecated modules were found. One was *optparse* and second was *logging.warn*. Unused import and one import being reimported were also found. Variable problems included an unused variable, name redefinition from an outer scope, possibly undefined loop variable, and cell variable defined in a loop.

Remaining problems were unnecessary *pass* statement, unnecessary *else* after *raise*, not re-raising using *from* keyword, too many arguments for a method, argument difference from an overridden method, *len* as a condition, unnecessary lambda, super *__init__* not being called, redefined built-in method, and a too broad exception.

In summary, Pylint offered a unique way of controlling various code parameters by defining a limit, focused mostly on problems with variables, function arguments and implementation, unnecessary code, and duplicated code.

3.2.2 DeepSource

DeepSource achieved the closest result. It reported 602 problems of 32 types.

Most issues were similar to those found by Pylint, with some slight differences. The most notable was a suggestion to use a literal syntax to create a data structure. This change can result in minor performance bumps.

Other less-frequent issues were the same implementation of an *if* branch, *next* not guarded with *try* block, *yield* not working as expected, test class with no tests, unneeded *del*, use of *getattr* without a default

value, comparison not using *in*, use of bare *except*, and named lambda expressions.

However, DeepSource has not found excisions of the maximum value for various parameters implemented by Pylint. The other missed issue was duplicated code in different files. Moreover, DeepSource ignored unused imports. The last not mentioned difference was not explicitly re-raising using *from* keyword in the report provided by DeepSource.

In summary, DeepSource focused mostly on problems with variables, function arguments and implementation, unnecessary code, and performance improvements.

3.2.3 SonarQube

SonarQube was again tested with two different settings. Testing with default settings resulted in 75 reported flaws of 12 types. Testing with all rules enabled brought a significant increase and resulted in 319 reported issues of 18 types.

Some problems were similar to those found by Pylint and DeepSource. These included too many statements, too many return statements, a method that could be function, unnecessary *pass* statement, cell variable defined in a loop, too broad exception, unused variable, the same implementation of an *if* branch, and duplicated blocks of code in different files. Some of the previously mentioned problems were only found with all rules enabled.

Nevertheless, numerous problems found by SonarQube were not mentioned by Pylint or DeepSource. Use of an insecure module *tempfile.mktemp*, duplication of string instead using a constant, exceeded cognitive complexity, not re-raising an exception to stop the program, present *FIXME* string, raising a generic exception, and multiple methods having the same implementation were problems discovered in both setting modes. Lastly, as the only tool, SonarQube reported three occurrences of a too complex file and a presence of *TODO* string.

In summary, SonarQube found multiple problems that were not found by other tools. It mainly focused on problems related to duplicated code, function implementation, and exceptions.

3.2.4 DeepCode

DeepCode found fewer issues than the previous three tools. It reported 46 problems of 7 types.

The reported problems consisted mostly of new types. The only two reoccurring issues were the same implementation of an *if* branch and *next* not guarded with *try* block.

The remaining problems were a use of deprecated module *tempfile.mktemp*, accessing attribute *__class__* on possibly *None* value, unneeded call of *list* function on a variable that is already a list, insecure module *xml.etree.ElementTree.parse*, and inability of *assertTrue* to provide an informative message if it fails.

In summary, DeepCode found few problems. Most of them were not found by other tools.

3.2.5 Pyflakes

Pyflakes reported the least types of problems. It reported 67 problems of 2 types. The first type of problem was an unused variable, and the second was an unused import.

3.3 Style

Style issues were among the most numerable problems. As mentioned before in Section 2.9, only Pylint, DeepSource and SonarQube provide the functionality to detect these flaws.

3.3.1 Pylint

The total number in this category reached 4,055 problems of 14 different types found by Pylint.

The most numerable issue was trailing whitespace or newline, invalid name, and missing or empty docstring. Import related issues included wrong import order, wrong position of an import, and import outside toplevel.

The rest of the issues found were unneeded *not*, missing final newline in a file, more than one statement on a single line, too long line, too many lines, and attribute defined outside of __init__.

3.3.2 DeepSource

DeepSource reported the most types of problems in this category. It found 2,674 problems of 21 types.

The most numerable issue was again trailing whitespace. Nevertheless, DeepSource found no problems related to docstrings, import order, import outside toplevel, or number of lines. The other difference was that only a few cases of an invalid name were reported.

According to the report, the project had various other flaws. These problems were only reported by DeepSource. They consisted of not using two blank lines between or after functions or classes, continuation line line missing indentation or outdented, missing blank line between methods, missing whitespaces around an operator, redundant backslash between brackets, block comment not having '#' at the beginning, multiple spaces around a keyword or an operator, less than two spaces before an inline comment, whitespaces before opening parenthesis, and missing whitespaces after ':', ';' and ','.

In summary, DeepSource performed similarly to Pylint. Moreover, a considerable amount of new style issues were found mostly related to the usage of blank lines and whitespaces.

3.3.3 SonarQube

Even with the base set of rules, SonarQube found three types of issues. These issues were mostly invalid names, some commented out pieces of code, and a single case of a useless parenthesis.

With the complete set of rules, nine additional types were revealed. The total number of issues was 2,666. As with the previous two tools, most of them were trailing whitespaces, invalid names, and missing or empty docstrings. Missing final newline, too long line, and too many lines were less frequent.

Comment at the end of a line, insufficient comment density, and lacking test coverage, along with the already mentioned presence of commented out pieces of code and useless parenthesis, were uniquely discovered by this tool.

In summary, SonarQube found some flaws already reported by DeepSource and Pylint. In addition, it also discovered new problems mainly related to comments and tests.

3.4 Execution times

Execution times of static analyses of this project were measured as in Section 2.20.

The fastest time was again achieved by Pyflakes. The analysis took 1.4 seconds. The second best time was achieved by DeepCode. This tool managed to analyze the whole project in 3.9 seconds. The third fastest tool was SonarQube. It finished in 18.2 seconds with default rules and in 20.5 seconds with all rules enabled. Pylint achieved the second-worst time. The whole analysis took 34.9 seconds. Finally, DeepSource was the slowest tool in this comparison. Testing with style checking enabled lasted for an average of 91.0 seconds.

Order of the tools based on execution time changed compared to results in Section 2.20. The highest ratio of execution time for the project to execution time for test cases can be seen in the case of Pylint. The project analysis took approximately 23.4 times more than the analysis of the test cases. The ratio of both times was checked by running both analyses subsequently. Whereas the ratio achieved by DeepCode and SonarQube was the lowest. In these cases, the project analysis took approximately 1.3 to 1.5 times more than the analysis of the test cases.

3.5 Summary

The selected project provided a sufficient base for the tested tools to demonstrate their potential. Altogether, testing revealed a few errors, multiple warnings and code smells, and a large number of style problems. No problem was found by all tools. Nevertheless, all tools except DeepCode agreed on three unused variables.

DeepSource found the widest variety of problems overall. Most of the uniquely found issues were style problems. However, it came at the cost that the static analysis took the longest time out of all tested tools.

Pylint found the second biggest number of different problems. Reports of Pylint and DeepSource were similar in many cases. However, Pylint also provided some uniquely found problems. The most notable feature was a distinctive way of controlling various code parameters

by defining limit values. Nevertheless, Pylint achieved the second-worst time. Notably, this time was still approximately two times faster than the time of DeepSource. As mentioned in Section 3.4, this tool demonstrated the worst ratio of the execution time for the project to execution time for test cases.

Results of SonarQube were largely dependent on the selection of the rules. With all rules enabled, it found two times more types of issues than with the base settings. This tool found many unique flaws. These flaws mainly consisted of code smells. The most notable was the code complexity evaluation method offered by this tool. Additionally, results have shown that this tool had an excellent ratio of execution time for the project to execution time for test cases.

DeepCode reported only seven different types of issues overall. Five types were unique. Nevertheless, the analysis finished with the second-best time. As in the case of SonarQube, DeepCode demonstrated an excellent ratio of execution time for the project to execution time for test cases.

The fewest types of issues were found by Pyflakes. This tool reported only unused imports and unused variables. Consequently, the analysis took the shortest time.

Table 3.1: Errors

	Pylint	Pyflakes	DeepSource	DeepCode	SonarQube default rules	SonarQube all rules
Number of issues	20	0	10	0	0	0
Types of issues	4	0	2	0	0	0
Import error	5	0	0	0	0	0
Instance having no member with the required name	4	0	0	0	0	0
No value for parameter	8	0	8	0	0	0
Object not being callable	3	0	2	0	0	0

There were no significant differences in the reported numbers.

Table 3.2: Warnings and code smells

	Pylint	Pyflakes	DeepSource	DeepCode	SonarQube default rules	SonarQube all rules
Number of issues	614	67	602	46	75	319
Types of issues	34	2	32	7	12	18
Accessing attribute __class__ on possibly None value	0	0	0	1	0	0
Argument difference from an overridden method	4	0	4	0	0	0
AssertTrue provides no info on failure	0	0	0	37	0	0
Cell variable defined in loop	1	0	1	0	1	1
Comparison not using in	0	0	6	0	0	0
Deprecated method warn	2	0	2	0	0	0
Deprecated optparse	7	0	7	0	0	0
Deprecated tempfile	0	0	0	1	0	0
Duplicated blocks of code	24	0	0	0	0	9
Duplication instead using a constant	0	0	0	0	14	14
Elif after return	27	0	27	0	0	0
Else after raise	2	0	2	0	0	0
Exceeded cognitive complexity	0	0	0	0	7	7
Exception caught being very general	1	0	1	0	1	1
If branches with the same implementation	0	0	1	2	3	3
Import being re-imported	1	0	1	0	0	0
Inconsistent return statements	20	0	20	0	0	0
Insecure tempfile	0	0	0	0	1	1
Insecure xml.etree.ElementTree.parse	0	0	0	1	0	0
Len used as a condition	5	0	5	0	0	0
Logging not being lazy	22	0	22	0	0	0
Method could be function	218	0	218	0	0	205
Named lambda expression	0	0	43	0	0	0
Next function not guarded with try-catch block	0	0	5	3	0	0
Not re-raising using from keyword	1	0	0	0	0	0
Not re-raising an exception to stop the application	0	0	0	0	2	2
Not using a literal syntax to create a data structure	0	0	48	0	0	0
Possibly undefined loop variable	1	0	1	0	0	0
Present FIXME	0	0	0	0	1	1
Present TODO	0	0	0	0	0	11
Raise of a generic exception	0	0	0	0	1	1
Redefined argument with a local name	98	0	98	0	0	0
Redefined built-in function	5	0	4	0	0	0
Redefined name from outer scope	4	0	4	0	0	0
Same implementation of a method or a function	0	0	0	0	31	31
Super __init__ not called	2	0	2	0	0	0
Test class without tests	0	0	2	0	0	0
Too complex file	0	0	0	0	0	3
Too few public methods	64	0	0	0	0	0
Too many ancestors	1	0	0	0	0	0
Too many arguments	9	0	0	0	0	0
Too many branches	4	0	0	0	0	0
Too many instance attributes	2	0	0	0	0	0
Too many local variables	4	0	0	0	0	0
Too many public methods	8	0	0	0	0	0
Too many return statements	2	0	0	0	0	11
Too many statements	1	0	0	0	0	5
Uneeded call of list function on a variable of type list	0	0	0	1	0	0
Uneeded del	0	0	1	0	0	0
Unnecessary lambda	2	0	2	0	0	0
Unnecessary pass	9	0	17	0	10	10
Unused argument	34	0	21	0	0	0
Unused import	3	64	0	0	0	0
Unused variable	3	3	3	0	3	3
Usage of bare except	0	0	1	0	0	0
Use of getattr without a default value	0	0	9	0	0	0
Useless object inheritance of object class	23	0	23	0	0	0
Yield function not working as expected	0	0	1	0	0	0

There are several issues that were found in quite different numbers by various tools. The most obvious example is unused import. Pylint counted the import as a whole. On the contrary, Pyflakes also reported distinctive classes and functions that were imported but unused.

Table 3.3: Style issues

	Pylint	Pyflakes	DeepSource	DeepCode	SonarQube default rules	SonarQube all rules
Number of issues	4055	0	2674	0	82	2666
Types of issues	14	0	21	0	3	12
At least two spaces before inline comment	0	0	1	0	0	0
Attribute defined outside __init__	11	0	10	0	0	0
Block comment missing #	0	0	60	0	0	0
Comment at the end of a line	0	0	0	0	0	7
Comment density	0	0	0	0	0	26
Commented out code	0	0	0	0	27	28
Continuation line missing indentation or outdented	0	0	36	0	0	0
Empty docstring	1	0	0	0	0	1
Import outside toplevel	3	0	0	0	0	0
Invalid name	1067	0	8	0	54	54
Line too long	18	0	200	0	0	18
Missing blank line between methods	0	0	1	0	0	0
Missing docstring	674	0	0	0	0	203
Missing final newline	1	0	1	0	0	2
Missing whitespace after ':', ';' and ','	0	0	4	0	0	0
Missing whitespaces around operator	0	0	2	0	0	0
Multiple spaces around keyword	0	0	27	0	0	0
Multiple spaces around operator	0	0	1	0	0	0
Multiple statements on one line	4	0	5	0	0	0
Redundant backslash between brackets	0	0	3	0	0	0
Test coverage	0	0	0	0	0	54
Too many lines	5	0	0	0	0	5
Trailing newlines	28	0	62	0	0	0
Trailing whitespaces	2225	0	2231	0	0	2233
Two blank line between functions or classes	0	0	6	0	0	0
Two blank lines after function or class	0	0	3	0	0	0
Unneeded not	1	0	1	0	0	0
Useless parentheses	0	0	0	0	1	35
Whitespaces before opening parenthesis	0	0	4	0	0	0
Wrong import order	9	0	0	0	0	0
Wrong import position	8	0	8	0	0	0

Some types of issues were merged into one category because of their similarity. Therefore, for example, useless parenthesis is significantly different between two SonarQube rule variants. With all rules enabled, SonarQube reported extra 34 cases of useless parenthesis after *return* keyword.

There are again several issues that were found in quite different numbers by various tools. These differences were caused by different definitions. For example, Pylint marked every name that did not conform to snake_case naming style as an invalid name. On contrary, DeepSource considered only *l*, *O* and *I* as invalid names. SonarQube had also its own definition of an invalid name. Every name needed to match a regular expression ^[*a-z_*][*a-z0-9_*]*$. Maximal line length had a different definition too. SonarQube had this limit set to 120 characters, Pylint to 100 characters and DeepSource to 88 characters.

4 General qualities

In addition to bug detection, various other factors may affect the user's experience. These factors are the main focus of the following sections. Tools are compared concerning their interface, issue documentation, community threads and price.

4.1 Interface

Based on the interface, all tested tools can be divided into two categories.

The first category consists of tools that are used through the command line. These tools are Pylint and Pyflakes. A user defines a file or a folder for analysis. Additionally, Pylint provides various options and switches [9]. Results of analyses are subsequently listed in the console by default. Pylint also assigns a score to every analysis. This score is then compared to the previous analysis of the file. Thanks to this feature, user can monitor how recent changes affected the overall quality of the code. Pyflakes does not provide similar options.

The second category consists of tools that use a web-based interface. DeepCode and DeepSource offer the interface through their web pages. SonarQube is dependent on a locally hosted server. In addition, SonarQube requires Oracle JRE 11 or OpenJDK 11 to be installed on the user's machine [18]. All interfaces in this category were clear and easy to use. In general, after authorization, a user can browse active projects, add new repositories, view issues, and set analysis options. DeepCode provided the fewest options. Analysis precision, report updates, and project branches were configurable. DeepSource and SonarQube offered the same options and added others [2, 14]. A notable example is Autofix in the case of DeepSource and rule management in the case of SonarQube. Furthermore, SonarQube and DeepSource keep track of previous analyses on the same project and compare them together.

Some tools can also provide different interfaces. However, this chapter focuses only on the interfaces that were used during the testing.

4.2 Issue documentation

Clarity of an issue report is another not negligible topic. DeepCode, DeepSource and SonarQube utilized the web-based interface and offered more information than the remaining two tools. Mentioned tools provided multiple sentences of description to every issue. Depending on the issue, these descriptions also sometimes featured correct and incorrect examples. The problematic line from the analyzed file was also highlighted. Moreover, there is one feature unique to SonarQube. This feature approximates the amount of time needed to fix a specific issue.

The base report generated by Pylint consisted of a file name, line and column of occurrence, code of an issue, and a brief description. Additionally, all issues are mentioned in the documentation [9]. This documentation provides additional information on all issues.

The most lacking were the reports of Pyflakes. These reports consisted of a file name, line and column of occurrence, and a brief description.

4.3 Community

Community threads with various bug reports provide valuable information to the developers. The majority of tested tools offer this feature. Pylint and Pyflakes feature a section on GitHub dedicated to this purpose [19, 20]. DeepSource and SonarQube also provide a similar section hosted on their websites [21, 22]. The only tool without the community threads is DeepCode. However, DeepCode offers an option to send feedback to the developers through the web-based interface. This feedback can also be used to report false positives by clicking the corresponding button.

4.4 Price

As already mentioned, all tested versions were free. However, some tools do have paid features.

DeepSource offers four different tiers [23]. Each tier is suited for a team of a different size. The first tier intended for small teams is free.

This tier offers an unlimited number of public repositories, maximum of three private repositories, unlimited Autofix sessions, a maximum of three team members, access to all analyzers, access to all transformers, and email support. Other tiers remove maximal restrictions on private repositories and the number of team members. The additional features available for higher tiers include priority support, advanced team management, manual invoicing, dedicated account manager, and on-premise installation. The price for the second tier is $12 per user per month. The highest tier with all features costs based on team size and usage.

SonarQube also offers four different tiers [24]. The first tier offers SonarQube and more than 60 plugins, SonarLint, and 15 languages. This tier is free and open source. Other tiers also offer branch analysis, pull request decoration, detection of injection flaws, SonarLint notifications, 27 languages, portfolio management and PDF executive reports, security reports, project transfer, parallel processing of analysis reports, component redundancy, data resiliency, and horizontal scalability. The price is dependent on lines of code. The cost for the second tier starts at €120 per year. The highest tier with all mentioned features costs at least €100,000.

DeepCode is currently free for public and private repositories and teams up to thirty developers [25].

5 Recommendation

Based on the acquired results, several recommendations can be made. These recommendations are dependent on the workflow of the particular user.

DeepSource is suitable for a small team that uses version control software. The main reason is a good performance, appealing interface, detailed issue documentation, support of version control systems, and no cost for a team smaller than four members. Optionally, a paid version of SonarQube can also be utilized. Unlike the free version of SonarQube, paid versions fully support the integration of version control software. This tool can prove helpful because it found a number of issues not reported by DeepSource in the project testing (Chapter 3).

Pylint and the free version of SonarQube are recommended for users that analyze their files locally. Pylint performed better than SonarQube in analyzing the test cases (Chapter 2) and found more issues during the project testing (Chapter 3). Whereas SonarQube has a better interface, issue documentation and found a number of issues not reported by Pylint during the project testing (Chapter 3).

However, if the top priorities are speed and the least amount of false positives, Pyflakes is the favourable choice. These priorities are the main focus of this tool [10], and the performed testing proved it. Nevertheless, this tool does not support version control software. Therefore, it is only useful for analyses on a user's computer.

DeepCode at the current state can not be recommended. However, as a tool based on AI, it has potential. Therefore, this tool may become relevant in the future.

Conclusion

The main goal of this thesis was to choose and evaluate different tools used for static analysis in Python.

The evaluation consisted of several test cases with known issues and an open-source project with more than 20,000 lines written in Python. Thanks to the unique scope of every test case, the results of different tools are compact and can be directly compared.

Subsequently, a comparison of the used tools was made based on the results acquired during the tests. The comparison focused on the number of reported issues, their severity, false positives, and user experience. Following the comparison, a final recommendation was made.

This thesis pointed out the suitability of DeepSource for a small teams that use version control software, Pylint for users that analyze files locally, SonarQube supplement to DeepSource or Pylint, and Pyflakes for fast analysis with the least amount of false positives. DeepCode can not be currently recommended.

The gained data can help teams or sole developers to find a suitable tool for their projects. Moreover, the specific bugs in the test cases (Chapter 2) and a sheer variety of issues in the open-source project (Chapter 3) allows distinguishing what types of problems each tool differentiates.

As stated in Chapter 1, this thesis is focused on universal tools. Nevertheless, the evaluation of tools used for a specific purpose can also be beneficial. Other possible continuations of this thesis include testing type control and evaluation of paid tools for teams larger than thirty developers.

Bibliography

1. AYEWAH, Nathaniel; PUGH, William; HOVEMEYER, David; MORGENTHALER, J. David; PENIX, John. Using Static Analysis to Find Bugs. *IEEE Software*. 2008, vol. 25, no. 5, pp. 22–29. Available from DOI: 10.1109/MS.2008.130.
2. *DeepSource: Automate code reviews with static analysis* [online] [visited on 2021-05-12]. Available from: https://deepsource.io/.
3. DEEPCODE, Marlene. *What is DeepCode?* [Online]. 2020 [visited on 2021-05-12]. Available from: https://deepcode.freshdesk.com/support/solutions/articles/60000346607-what-is-deepcode-.
4. *vulture – PyPI* [online]. 2021 [visited on 2021-05-12]. Available from: https://pypi.org/project/vulture/.
5. GULABOVSKA, Hristina; PORKOLÁB, Zoltán. *Survey on Static Analysis Tools of Python Programs* [online]. Eötvös Loránd University, 2019 [visited on 2021-05-12]. Available from: http://ceur-ws.org/Vol-2508/paper-gul.pdf.
6. VANTOL, Alexander. *Python Code Quality: Tools & Best Practices* [online] [visited on 2021-05-12]. Available from: https://realpython.com/python-code-quality/.
7. DEWAN, Chiragh. *SonarQube: A Hidden Gem* [online]. InfoSec Institute, 2017 [visited on 2021-05-12]. Available from: https://resources.infosecinstitute.com/topic/sonarqube-hidden-gem/.
8. *DeepCode package for Atom* [online] [visited on 2021-05-12]. Available from: https://atom.io/packages/deepcode.
9. *Pylint User Manual* [online] [visited on 2021-05-12]. Available from: http://pylint.pycqa.org/en/latest/.
10. *Pyflake GitHub repository* [online] [visited on 2021-05-12]. Available from: https://github.com/PyCQA/pyflakes.
11. *DeepCode solutions* [online] [visited on 2021-05-12]. Available from: https://deepcode.freshdesk.com/support/solutions.

12. *History | SonarSource* [online] [visited on 2021-05-12]. Available from: https://www.sonarsource.com/company/history/.

13. *Code Quality and Code Security | SonarQube* [online] [visited on 2021-05-12]. Available from: https://www.sonarqube.org/.

14. *SonarQube Documentation | SonarQube Docs* [online] [visited on 2021-05-12]. Available from: https://docs.sonarqube.org/latest/.

15. *Python library for parsing, manipulating, and generating BridgePoint xtUML models* [online] [visited on 2021-05-12]. Available from: https://github.com/john-tornblom/pyxtuml.

16. TÖRNBLOM, John. *Getting Started – pyxtuml 2.2.1 documentation* [online]. 2016 [visited on 2021-05-12]. Available from: https://pyxtuml.readthedocs.io/en/master/getting-started.html.

17. TÖRNBLOM, John. *Project commit* [online]. 2018 [visited on 2021-05-12]. Available from: https://github.com/john-tornblom/pyxtuml/commit/7dd9343b9a0191d1db1887ab9288d0a026608d9a.

18. *Try Out SonarQube | SonarQube Docs* [online] [visited on 2021-05-12]. Available from: https://docs.sonarqube.org/latest/setup/get-started-2-minutes/.

19. *Issues – PyCQA/pylint* [online] [visited on 2021-05-12]. Available from: https://github.com/PyCQA/pylint/issues.

20. *Issues – PyCQA/pyflakes* [online] [visited on 2021-05-12]. Available from: https://github.com/PyCQA/pyflakes/issues.

21. *DeepSource Community Forum* [online] [visited on 2021-05-12]. Available from: https://discuss.deepsource.io/.

22. *Latest Report a bug topics - SonarSource Community* [online] [visited on 2021-05-12]. Available from: https://community.sonarsource.com/c/bug/5.

23. *Pricing – DeepSource* [online] [visited on 2021-05-12]. Available from: https://deepsource.io/pricing/.

24. *Plans & Pricing | SonarSource* [online] [visited on 2021-05-12]. Available from: https://www.sonarsource.com/plans-and-pricing/.

25. *Developers love us* [online] [visited on 2021-05-12]. Available from: https://www.deepcode.ai/cloud-login.

A File to section mapping

Filename	Section
argument_number_always_false.py	2.1 Invalid number of arguments and 2.2 Ineffective logic formulas
array_access.py	2.3 Array access
assign.py	2.4 Equals sign misuse
attribute.py	2.15 Function argument type
attribute_2.py	2.15 Function argument type
built_in_functions.py	2.1 Invalid number of arguments
division_by_zero.py	2.6 Possibility of division by zero
division_by_zero_2.py	2.6 Possibility of division by zero and 2.9 Style issues
division_by_zero_3.py	2.6 Possibility of division by zero and 2.9 Style issues
empty_range.py	2.1 Invalid number of arguments
file_manipulation.py	2.5 File manipulation
key.py	2.7 Invalid dictionary keys
membership_support.py	2.8 Membership support
membership_support_fp.py	2.8 Membership support
missing_docfile.py	2.9 Style issues
overflow_float.py	2.10 Variable overflow
possibly_undefined_fn.py	2.17 Possibly undefined variable
recursion.py	2.11 Recursion
recursion_class_method.py	2.11 Recursion
undefined_function_later_defined.py	2.13 Undefined functions
undefined_variable.py	2.12 Undefined variables
unreachable_code.py	2.14 Unreachable code
unsubsriptable_fp.py	2.16 Changed type of variable
unused_import.py	2.18 Unused or missing import